# EXPERIMENT NO.11

**Name:**Vaishnal Mali
**Class**:D20A
**Roll No**:32
**Batch**:B

**Aim:**To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

**Prerequisites:**

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

**Procedure:**

**Step 1: Update and Install dependencies**

**Command:**

```
sudo apt update
sudo apt upgrade -y
```

```
vaishnal@Vaishnal:~$ sudo apt update
sudo apt upgrade -y
sudo apt install jq curl git nodejs npm -y
[sudo] password for vaishnal:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1579 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [253 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1170 kB]
```

**Step 2: Clone Fabric Samples**

**Command:**

git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples

```
vaishnal@Vaishnal:~$ git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 390.00 KiB/s, done.
Resolving deltas: 100% (8809/8809), done.
vaishnal@Vaishnal:~/fabric-samples$
```

**Step 3: Install Fabric**

**Command:**

curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

```
vaishnal@Vaishnal:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install
-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent ve
rsions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15
.tar.gz
===> Will unpack to: /home/vaishnal/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0   487k      0  0:04:13  0:04:13 --:--:--  397k
```

## Step 4: Verify Installation

## Command:

docker images | grep fabric

```
vaishnal@Vaishnal:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```

```
Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====>  ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====>  ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
```

## Step 5: Set Environment Variables

## Command:

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```
vaishnal@Vaishnal:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
vaishnal@Vaishnal:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

vaishnal@Vaishnal:~/fabric-samples$
```

## Step 6: Start Network

## Command:

cd ~/fabric-samples/test-network
./network.sh up createChannel

```
vaishnal@Vaishnal:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using da
ldb with crypto from 'cryptogen'
Bringing up network

LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/vaishnal/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
 ✔Network fabric_test                   Created
 ✔Volume compose_peer0.org1.example.com Created
```

```
++ configtxlator proto_encode --input /home/vaishnal/fabric-samples/test-network/channel-artifacts/config_update_in_en
lope.json --type common.Envelope --output /home/vaishnal/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.
2026-04-09 17:04:04.975 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-09 17:04:04.990 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
vaishnal@Vaishnal:~/fabric-samples/test-network$
vaishnal@Vaishnal:~/fabric-samples/test-network$
```

## Step 7: Deploy Chaincode

## Command:

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```
vaishnal@Vaishnal:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javas
basic_1.0
```

**Step 8: Set Peer Environment**

**Command:**

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"

export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

```
vaishnal@Vaishnal:~/fabric-samples/test-network$ export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.c
om/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/m
sp
export CORE_PEER_ADDRESS=localhost:7051
```

## Step 9: Initialize Ledger

### Command:

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers
/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}'
```

```
vaishnal@Vaishnal:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac
t.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}'
2026-04-09 17:26:44.205 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
vaishnal@Vaishnal:~/fabric-samples/test-network$
```

## Step 10: Query Assets

### Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"
Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3
","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Siz
e":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

**Step 11: Create Asset**

**Command:**

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"CreateAsset","Args":["asset9","blue","10","Manav","500"]}'
```

```
vaishnal@Vaishnal:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"CreateAsset","Args":["asset9","blue","10","Manav","500"]}'
2026-04-09 17:31:52.427 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset9\
e\",\"Size\":10,\"Owner\":\"Manav\",\"AppraisedValue\":500}"
vaishnal@Vaishnal:~/fabric-samples/test-network$
```

**Step 12: Transfer Asset**

**Command:**

peer chaincode invoke ... TransferAsset

```
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
2026-04-03 13:41:46.421 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
bacafe5ad0d3aaadd] committed with status (VALID) at localhost:9051
2026-04-03 13:41:46.424 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
bacafe5ad0d3aaadd] committed with status (VALID) at localhost:7051
2026-04-03 13:41:46.426 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:"Manav"
```

## Step 13: Stop Network

## Command:

./network.sh down

```
vaishnal@Vaishnal:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container peer0.org1.example.com      Removed
 ✔Container peer0.org2.example.com      Removed
 ✔Container orderer.example.com         Removed
 ✔Volume compose_orderer.example.com    Removed
 ✔Volume compose_peer0.org1.example.com Removed
 ✔Volume compose_peer0.org2.example.com Removed
 ✔Network fabric_test                   Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
c5ea452fab11
50ae609fc026
Removing generated chaincode docker images
```

## Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

**Conclusion:**

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.